

Problem A — Dense vs. Sparse Matrix-Vector Multiplication

Setup

Computation environment, fixed throughout: Intel Core Ultra 5 225H (14 cores), 32 GiB DDR5, Linux 7.0.1, Python 3.13.12, NumPy 2.4.4, SciPy 1.17.1, scipy-openblas 0.3.31 with all cores enabled. The matrix dimension is fixed at $N = 4000$; A is the output of `init_matrices(N, p)` and x is a fixed `float64` vector of length N . Each timing is the minimum of 50 timed $A @ x$ runs after 3 warm-up runs. The Python construction loop inside `init_matrices` runs only once and is excluded from the timing, consistent with the stated assumption that the target application multiplies the same matrix.

A-1 — Benchmark and discussion

The benchmark driver `problem_a1.py` sweeps the non-zero density p and times $A_{\text{dense}} @ x$ against $A_{\text{sparse}} @ x$ (CSR). Selected rows of the resulting `benchmark_results.json` are reproduced below.

p	nnz	dense (ms)	sparse (ms)	speed-up
0.001	16 051	1.60	0.014	$\times 118$
0.010	160 225	1.73	0.131	$\times 13$
0.050	802 034	1.73	0.756	$\times 2.3$
0.100	1 601 646	1.58	1.21	$\times 1.3$
0.150	2 402 221	1.58	1.80	$\times 0.88$
0.200	3 202 938	1.64	2.28	$\times 0.72$

Dense throughput reaches ~ 20 GFLOP/s; CSR throughput reaches ~ 2.7 GFLOP/s for $p \geq 0.05$.

Throughput analysis. Both kernels are bounded by memory bandwidth rather than by floating-point throughput. The dense call dispatches to OpenBLAS `dgemv`, which streams every entry of A : the matrix occupies $N^2 \cdot 8 \text{ B} = 128 \text{ MiB}$, so reading it in 1.6 ms corresponds to approximately **80 GB/s** — close to the practical DDR5 ceiling of this machine ($\sim 90 \text{ GB/s}$). The CSR (Compressed Sparse Row) format stores only the non-zero entries (one 8-byte value plus one 4-byte column index per non-zero, plus a small row-pointer array); the SciPy implementation is in C but single-threaded. At $p \geq 0.05$ it sustains approximately 16 GB/s, the bandwidth a single core can extract from one DDR5 channel. Both kernels therefore operate at their respective hardware limits, and the dense-vs-CSR gap is primarily a consequence of the dense kernel exploiting all 14 cores while CSR uses one.

Effect of density on performance.

1. Dense execution time is essentially independent of p (≈ 1.6 ms throughout): multiplications by zero entries consume the same memory bandwidth and floating-point cycles as non-zero entries, so zeros are not free under the dense format.
2. CSR execution time grows linearly with the number of non-zeros; doubling p doubles the time (1.21 ms at $p = 0.1 \rightarrow 2.28$ ms at $p = 0.2$).
3. The crossover lies at approximately $p \approx 0.13$ (15% density). Beyond this point the 4-byte column index per non-zero plus the absence of multi-threading in the CSR kernel outweigh the saving from skipping zero entries.
4. The crossover is sensitive to thread count. Setting `OPENBLAS_NUM_THREADS=1` raises the dense time to 3.9 ms and pushes the crossover beyond $p = 0.2$, so the appropriate format also depends on the BLAS thread configuration.

Memory footprint similarly favours CSR: $\approx 12pN^2$ bytes versus the dense $8N^2$, so CSR is the smaller representation up to roughly $p \approx \frac{2}{3}$.

A-2 — PageRank as an SpMV-bound application

Definition. PageRank assigns to each page i of a directed web graph a numerical importance $x[i]$, defined as the stationary distribution of a **random surfer** who at each step either follows a uniformly random outgoing link (with probability α) or teleports to a uniformly random page (with probability $1 - \alpha$). The teleport term, parameterised by the **damping factor** $\alpha = 0.85$, makes the underlying Markov chain irreducible and aperiodic — so the stationary distribution is unique — and bounds the spectral gap, which controls the convergence rate of the iterative solver. With P the row-stochastic transition matrix of the link graph, the rank vector satisfies the fixed-point equation

$$x = \alpha \cdot P^T x + \frac{1 - \alpha}{N} \cdot \mathbf{1}.$$

Reduction to repeated SpMV. The standard solver is the **power iteration**: starting from $x^{(0)} = \mathbf{1}/N$, the right-hand side above is applied repeatedly. Each iteration is one CSR multiplication by the fixed matrix P^T plus an $O(N)$ vector update; the residual shrinks by a factor of α per iteration, so 50–100 iterations reach a tolerance of 10^{-8} to 10^{-10} . Real web graphs have only 10–30 outgoing links per page regardless of N , so at $N = 10^8$ the matrix has $\text{nnz} \approx 3 \cdot 10^9$ (36 GB in CSR) while the dense representation would require 10^{16} entries and is not a viable storage option — the operating point is the very-low-density regime of Section A-1, where CSR is already 100–400 times faster than dense.

Measured cost. The script `problem_a2.py` runs the power iteration above using the SpMV kernel of Section A-1, with $\mathbf{P}$ taken to be the row-normalised output of `init_matrices`, $N = 4000$, $\alpha = 0.85$, tolerance 10^{-10} , and at most 200 iterations.

p	iters	total SpMV (ms)	per iter (μ s)	SpMV / total
$1 \cdot 10^{-3}$	93	1.33	14	63%
$5 \cdot 10^{-3}$	16	0.79	50	84%
$1 \cdot 10^{-2}$	13	1.77	137	88%
$1 \cdot 10^{-1}$	8	14.36	1795	98%

The SpMV share rises from 63% to 98% as the matrix becomes denser; the vector update and renormalisation are negligible whenever the matrix is non-trivial, so the throughput measured in Section A-1 directly predicts the application throughput. Extrapolating to $N = 10^7$ with average out-degree 20 ($\text{nnz} = 2 \cdot 10^8$), the CSR kernel of Section A-1 yields 1.5 s per SpMV; 80 power iterations therefore cost roughly 2 minutes of wall time — practical, but with all non-SpMV work invisible.

SpMV specialisations for the PageRank matrix

1. **Binary-matrix SpMV (factor out the row-stochastic scaling).** Every non-zero of P has the value $1/d_{\text{out}(i)}$, which depends only on the row, so $P^T = A^T D$ where $A \in \{0, 1\}^{N \times N}$ is the binary link adjacency and $D = \text{diag}(1/d_{\text{out}})$. The PageRank step becomes $P^T x = A^T (Dx) = A^T z$ with $z = Dx$ pre-computed in $O(N)$. The SpMV then reads only column indices, no values, so bytes per non-zero drop from 12 to 4 — a $3 \times$ reduction in memory traffic on this memory-bound kernel. The decomposition is unavailable to a generic SpMV because arbitrary matrix entries are not products of a row-only factor and a binary indicator.
2. **Reorder columns by host, then delta-encode indices.** Real web graphs have 80% of links staying within the same host. After permuting page IDs so that pages on the same host are contiguous, the column indices within each row of P^T are tightly clustered — typically within a span of a few thousand — so CSR can store column **deltas** in 16 bits instead of absolute 32-bit indices, shrinking index bytes by 2–4 $\times$. Combined with item 1 the SpMV reads 2 B per non-zero. The reordering is paid once and reused; the speed-up depends on a property of web graphs, not of sparse matrices in general.
3. **Hub-row chunking for power-law in-degree.** In-degree on a web graph follows a power law: a few pages have 10^5+ in-links while the median has under 10. A row-partitioned parallel CSR SpMV gives catastrophic thread imbalance — the thread owning a hub row works thousands of times longer than its neighbours. A PageRank-tuned kernel splits any row longer than a threshold (chosen at the inflection point of the measured in-degree distribution) into chunks that several threads sum into the same $y[j]$ via partial accumulators. Uniform-padding formats (ELLPACK / Sliced-ELLPACK) cannot do this without wasting enormous storage on the long tail.
4. **Fused SpMV + dangling correction + AXPY.** The full PageRank step is $x \leftarrow \alpha(P^T x) + (\alpha s + 1 - \alpha)/N \cdot \mathbf{1}$ with $s = \sum_{i \in D} x[i]$ over dangling pages D . A textbook pipeline performs this in three separate passes over y (SpMV, scale, add bias). A specialised kernel emits each $y[j]$ in a single streaming pass over CSR — row sum, scale by α , add the pre-computed scalar bias — and updates s for the next iteration on the same pass. This roughly halves the per-iteration y -traffic on the memory-bound kernel.